SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME:Artificial Intelligence COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.*(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 3 Duration : 1 Hour

Total Marks:30

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I D. Jeshurun Silas (192525324) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: D. Jeshurun Silas

Annexure A

Constraint-Based Problem Solving

1. A hospital needs to assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night) such that: Constraints:

i. D1 cannot work Night shift ii. D2 must work before D3 (Morning < Afternoon < Night) iii. Only one doctor per shift iv. D3 cannot work Morning

v. All doctors must be assigned exactly one shift

Apply Backtracking Search to find a valid assignment with all intermediate steps and constraint checks. State the final valid schedule

2. A robot operates in a grid environment (5×5). It must move from Start (S) to Goal (G). Obstacles block certain paths.

Constraints:

i. Movement allowed: Up, Down, Left, Right ii. Cost of each move = 1 iii. Cannot pass through obstacles iv. Use Manhattan Distance heuristic

Using above constraints and BFS Search Algorithm Show step-by-step node expansion and priority queue updates and determine the optimal path and cost**.**

3. An autonomous rescue robot is deployed in a disaster-affected building to locate and rescue survivors. The robot operates in a partially observable environment where some paths are blocked, and it must make decisions with limited information.

The building is represented as a grid of rooms. The robot starts at position S and must reach a survivor located at G.

Constraints:

- The robot can move up, down, left, right
- Some cells are blocked (obstacles) and cannot be traversed
- The robot has limited sensing capability (can only observe adjacent cells)
- Each movement has a cost of 1, but entering risky zones has an additional cost of 2
- The robot must minimize total cost while ensuring safe navigation

- The robot must update its knowledge dynamically (online search scenario)

Tasks:

a) Analyze all the given constraints and explain how they affect the robot's decision-making process.

b) Develop a solution approach using an appropriate search strategy (e.g., Uniform Cost Search / Online Search Agent). Clearly explain the steps involved. c) Demonstrate how the robot applies AI concepts such as:

- Intelligent agents
- Rationality
- Environment type

d) Justify why your chosen approach is the most suitable for solving this problem under the given constraints.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	25%	Clearly defines the problem and identifies all relevant constraints accurately	Identifies most constraints with minor gaps	Partial understanding; misses key constraints	Fails to identify constraints correctly
Constraint Analysis	25%	Analyzes constraints effectively and prioritizes them logically	Provides reasonable analysis with some prioritization	Limited analysis; weak prioritization	No meaningful analysis or prioritization
Solution Development	25%	Develops optimal and feasible solutions satisfying all constraints	Develops workable solutions with minor limitations	Solutions partially satisfy constraints	Solutions do not meet constraints
Application of Concepts	15%	Effectively applies appropriate concepts, methods, or tools	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply relevant concepts

Justification	10%	Provides strong justification for decisions with logical reasoning	Provides justification with some clarity	Weak or unclear justification	No justification provided

Introduction and Theoretical Background

This report solves three constraint-based problems with core uninformed and informed search methods from the Artificial Intelligence syllabus (CO1). Each solution gives the formulation, a full trace, a verified result and an analysis of why the method works.

Purpose and Scope

The assessment checks whether a student can model a real situation as a search or constraint problem and then solve it with a suitable algorithm. The three questions move from a small scheduling problem, to path planning on a known grid, to planning with incomplete information.

Question	Problem type	Technique applied	Key skill tested
Q1	Hospital shift scheduling	Backtracking search (CSP)	Modelling variables, domains and constraints
Q2	Grid path planning	BFS and A* with Manhattan heuristic	Node expansion, queue handling, optimal cost
Q3	Disaster rescue robot	Online uniform cost search with re-planning	Agent design, rationality, environment analysis

Intelligent Agents

An **agent** perceives its environment through sensors and acts on it through actuators. A **rational agent** chooses the action expected to maximise its performance measure, given its percepts and built-in knowledge. Agents differ in how they use memory and goals.

Agent type	How it decides	Example in this report
Simple reflex	Condition–action rules on the current percept only	Not suitable: the rescue robot needs memory
Model-based reflex	Keeps an internal model of the world	The robot's known map in Q3

Agent type	How it decides	Example in this report
Goal-based	Searches for actions that reach a goal	The BFS and A* planners in Q2
Utility-based	Maximises a utility or minimises a cost	UCS with risk costs in Q3
Learning	Improves its model or policy from experience	Map updating during online search

Search Strategies Compared

Search algorithms are judged on four properties: **completeness** (finds a solution if one exists), **optimality** (finds the cheapest solution), **time** and **space**. Here b is the branching factor, d the depth of the shallowest goal, m the maximum depth, l a depth limit and C* the optimal cost.

Algorithm	Type	Complete	Optimal	Time	Space
Breadth-First Search (BFS)	Uninformed	Yes (finite b)	Yes, for unit step costs	O(b^d)	O(b^d)
Depth-First Search (DFS)	Uninformed	No (loops, infinite depth)	No	O(b^m)	O(bm)
Depth-Limited Search	Uninformed	No (if $l < d$)	No	O(b^l)	O(bl)
Iterative Deepening DFS	Uninformed	Yes	Yes, for unit step costs	O(b^d)	O(bd)
Uniform Cost Search (UCS)	Uninformed	Yes (costs $\geq \varepsilon > 0$)	Yes	O(b^(1+C*/ε))	O(b^(1+C*/ε))
Greedy Best-First	Informed	No	No	O(b^m)	O(b^m)
A*	Informed	Yes	Yes, with an admissible heuristic	Exponential in the worst case, usually far less	Exponential in the worst case

BFS expands nodes level by level using a **FIFO queue**. UCS expands the node with the lowest path cost g(n) using a **priority queue**. A* orders its priority queue by f(n) = g(n) + h(n), so it is UCS guided by a heuristic.

$$f(n) = g(n) + h(n)$$

$$h_{\text{Manhattan}}(r, c) = |r - r_{goal}| + |c - c_{goal}|$$

Constraint Satisfaction Problems

A **CSP** is defined by a set of variables, a domain of values for each variable and a set of constraints that restrict which combinations are allowed. A solution is a complete assignment that satisfies every constraint.

Concept	Meaning	Used in Q1 as
Variable	An item that needs a value	D1, D2, D3
Domain	The values a variable may take	{Morning, Afternoon, Night}
Unary constraint	Restricts one variable	D1 ≠ Night, D3 ≠ Morning
Binary constraint	Restricts a pair of variables	D2 before D3, All-Different
Backtracking search	Depth-first assignment with constraint checks and undo on failure	Main method
Forward checking	After each assignment, remove inconsistent values from unassigned domains	Extended analysis, section 1.8
MRV heuristic	Choose the variable with the fewest remaining values next	Extended analysis, section 1.8
Arc consistency	Every value of one variable has a supporting value in a related variable	Extended analysis, section 1.8

Rubric Mapping

The assessment rubric has five criteria. The table shows where this report addresses each one.

Rubric criterion	Weight	Where it is addressed
Problem understanding	25%	Sections 1.1, 2.1, 3.0 and 3.1: formulation of states, actions and constraints
Constraint analysis	25%	Sections 1.1, 1.7, 1.10, 2.1, 2.9 and 3(a): constraint types, priorities and what-if analysis
Solution development	25%	Sections 1.3, 2.2, 2.3, 3(b) and 3.3: complete traces and verified optimal results
Application of concepts	15%	Introduction, 1.8, 2.6–2.8, 3(c) and 3.4: algorithms, heuristics, agent theory
Justification	10%	Sections 2.5, 3(d) and 3.5: comparison with alternative methods

Conventions Used

- Cells are written as (row, column), counted from 0 at the top left.
- Moves are tried in the order Up, Down, Left, Right.
- M, A and N stand for Morning, Afternoon and Night. The order is M < A < N.
- A tick ✓ means a constraint holds and a cross ✗ means it is violated.

Question 1 – Hospital Shift Scheduling using Backtracking Search

1.1 Problem Understanding

This is a **Constraint Satisfaction Problem (CSP)** with three parts.

CSP Component	Definition
Variables	D1, D2, D3 (each doctor needs a shift)
Domain	{Morning (M), Afternoon (A), Night (N)} for each variable

CSP Component	Definition
Time order	M < A < N

Constraints

No.	Constraint	Type
C1	D1 ≠ Night	Unary
C2	D2 must work before D3, i.e. shift(D2) < shift(D3)	Binary
C3	Only one doctor per shift (all shifts different)	Binary (All-Different)
C4	D3 ≠ Morning	Unary
C5	Every doctor gets exactly one shift	Assignment completeness

Constraint analysis. The unary constraints C1 and C4 are the cheapest to check, so they are tested first. C3 (All-Different) is then checked at each assignment. C2 is checked once both D2 and D3 are assigned. Pruned domains: D1 ∈ {M, A}, D2 ∈ {M, A, N}, D3 ∈ {A, N}.

1.2 Backtracking Search – Setup

- **Variable order:** D1 → D2 → D3
- **Value order:** Morning → Afternoon → Night
- **Method:** assign a value, check all constraints that are fully determined, and backtrack on any violation.

1.3 Step-by-Step Trace

Step	Assignment attempted	Constraint check	Result
1	D1 = Morning	C1 (D1 ≠ N): ✓	**Accept**, go deeper
2	D2 = Morning	C3: Morning already used by D1 ✗	**Reject**, try next value
3	D2 = Afternoon	C3: ✓	**Accept**, go deeper
4	D3 = Morning	C4 (D3 ≠ M) ✗ and C3 ✗	**Reject**, try next value

Step	Assignment attempted	Constraint check	Result
5	D3 = Afternoon	C3: Afternoon used by D2 ✗	**Reject**, try next value
6	D3 = Night	C4 ✓, C3 ✓, C2: Afternoon < Night ✓, C5 ✓	**All constraints satisfied, goal reached**

1.4 Search Tree

Root (no assignment)
└── D1 = Morning ✓
  ├── D2 = Morning ✗ (C3)
  ├── D2 = Afternoon ✓
  │ ├── D3 = Morning ✗ (C4, C3)
  │ ├── D3 = Afternoon ✗ (C3)
  │ └── D3 = Night ✓ GOAL (D1=M, D2=A, D3=N)
  └── D2 = Night (not explored, solution already found)

1.5 Final Valid Schedule

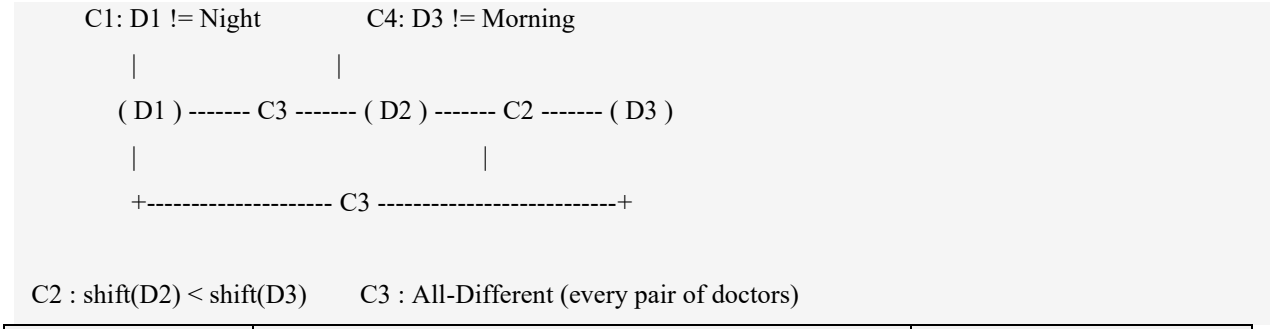
Shift	Doctor
🌅 Morning	**D1**
🌞 Afternoon	**D2**
🌙 Night	**D3**

Verification: C1 ✓ (D1 is on Morning), C2 ✓ (D2 Afternoon < D3 Night), C3 ✓ (one doctor per shift), C4 ✓ (D3 is not on Morning), C5 ✓ (each doctor has exactly one shift).

Note on completeness. If the search continues after this solution, it also finds a second valid schedule: D1 = Afternoon, D2 = Morning, D3 = Night. D1 = Night is pruned by C1. Backtracking returns the first valid schedule, **D1–Morning, D2–Afternoon, D3–Night**, because of the chosen variable and value ordering.

1.6 Constraint Graph

The constraint graph has one node per variable and one edge per constraint between two variables. Unary constraints are written beside the node.



Edge	Constraint	Checked when
D1 – D2	C3 (different shifts)	D2 is assigned
D1 – D3	C3 (different shifts)	D3 is assigned
D2 – D3	C3 and C2 (D2 earlier than D3)	D3 is assigned
D1 (unary)	C1	D1 is assigned
D3 (unary)	C4	D3 is assigned

D2 and D3 are connected by two constraints, so that pair is the most constrained part of the problem.

1.7 Exhaustive Enumeration of All Assignments

With 3 doctors and 3 shifts there are 3! = 6 different one-to-one assignments. Checking all six confirms that the backtracking result is complete.

No.	D1	D2	D3	C1 (D1 ≠ N)	C2 (D2 < D3)	C4 (D3 ≠ M)	Valid?
1	M	A	N	✓	✓ (A < N)	✓	**Yes**
2	M	N	A	✓	✗ (N > A)	✓	No
3	A	M	N	✓	✓ (M < N)	✓	**Yes**
4	A	N	M	✓	✗ (N > M)	✗	No
5	N	M	A	✗	✓ (M < A)	✓	No

No.	D1	D2	D3	C1 (D1 ≠ N)	C2 (D2 < D3)	C4 (D3 ≠ M)	Valid?
6	N	A	M	✗	✗ (A > M)	✗	No

Exactly **two** valid schedules exist: No. 1 (D1 Morning, D2 Afternoon, D3 Night) and No. 3 (D2 Morning, D1 Afternoon, D3 Night). In both, D3 works Night. Backtracking with the chosen ordering returns No. 1 first.

Effort comparison (full search for all solutions, no forward checking)

Method	Assignments examined
Unpruned tree (3 + 9 + 27 nodes)	39
Backtracking with constraint checks	21

Backtracking examines 21 of the 39 possible assignments because it abandons a branch as soon as a constraint fails. The branch D1 = Night is cut immediately by C1.

1.8 Forward Checking, MRV and Arc Consistency

Backtracking can be made stronger. **Forward checking** removes inconsistent values from the domains of unassigned variables after each assignment. **MRV** picks the variable with the smallest domain, and **arc consistency** removes values that have no supporting value in a related variable.

Step 1: apply the unary constraints (node consistency).

Variable	Initial domain	After C1 and C4
D1	{M, A, N}	{M, A}
D2	{M, A, N}	{M, A, N}
D3	{M, A, N}	{A, N}

Step 2: search with MRV and forward checking. D1 and D3 both have two values, and D1 is chosen first because it is tried first in value order.

Step	Action	Domains after propagation	Note
1	Assign D1 = M	D2 {A, N}, D3 {A,	M removed from D2 and D3 by C3

Step	Action	Domains after propagation	Note
		N}	
2	Arc consistency on C2 (D2 < D3)	D2 {A}, D3 {N}	D2 = N has no later shift, so it is removed. D3 = A has no earlier D2, so it is removed.
3	Assign D2 = A (only value)	D3 {N}	No failure
4	Assign D3 = N (only value)	complete	All constraints satisfied

With forward checking and arc consistency the solver finds D1 = M, D2 = A, D3 = N with **zero backtracking steps**, compared with 3 rejected assignments in the plain backtracking trace (steps 2, 4 and 5 in section 1.3). This shows that propagation does the work that trial and error does in the basic method.

1.9 Pseudocode and Complexity

```
function BACKTRACKING-SEARCH(csp):
  return BACKTRACK({}, csp)

function BACKTRACK(assignment, csp):
  if assignment is complete: return assignment
  var <- SELECT-UNASSIGNED-VARIABLE(csp, assignment)     // D1, D2, D3 order
  for each value in ORDER-DOMAIN-VALUES(var, csp):       // M, A, N order
    if value is consistent with assignment under all constraints:
      add {var = value} to assignment
      result <- BACKTRACK(assignment, csp)
      if result != failure: return result
      remove {var = value} from assignment          // backtrack
  return failure
```

Measure	Value for this problem
Number of variables n	3
Domain size d	3
Worst-case complete assignments	d^n = 27

Measure	Value for this problem
Maximum search depth	3 (one level per doctor)
Time complexity (general CSP)	O(d^n) in the worst case
Space complexity	O(n), since only the current assignment is stored
Complete?	Yes: all assignments are reachable, so a solution is found if one exists

1.10 What-If Analysis of the Constraints

Removing one constraint at a time shows which constraints matter most. The counts come from the enumeration table in section 1.7.

Constraint removed	Valid schedules left	Interpretation
None (original problem)	2	Nos. 1 and 3
C1 (D1 ≠ Night)	3	Adds No. 5 (D1 Night, D2 Morning, D3 Afternoon)
C2 (D2 before D3)	3	Adds No. 2 (D1 Morning, D2 Night, D3 Afternoon)
C4 (D3 ≠ Morning)	2	No change, so C4 is **redundant**: if D2 must work before D3, D3 can never be on Morning

The most restrictive constraints are C1 and C2, because each one removes a valid schedule when it is present. C4 adds no extra restriction once C2 is in force, which is useful to know when a hospital wants to simplify its rule set. Constraint priority for checking is therefore: C1 and C4 (unary, cheapest), then C3, then C2.

Question 2 – Robot Grid Navigation (BFS with Manhattan Distance)

2.1 Problem Understanding and Assumptions

The question does not include the grid figure, so the following **5×5 grid is assumed** (coordinates are (row, column), starting from (0,0) at the top left).

	c0	c1	c2	c3	c4
r0	🤖 S	⬜	⬜	⬛	⬜
r1	⬜	⬛	⬜	⬛	⬜
r2	⬜	⬛	⬜	⬜	⬜
r3	⬜	⬜	⬜	⬛	⬜
r4	⬛	⬜	⬜	⬜	🏁 G

Legend: 🤖 S = Start (0,0), 🏁 G = Goal (4,4), ⬛ = Obstacle, ⬜ = Free cell. **Obstacles:** (0,3), (1,1), (1,3), (2,1), (3,3), (4,0).

Problem formulation

Element	Definition
State	Robot position (row, col)
Initial state	(0,0)
Actions	Up, Down, Left, Right (expanded in this order)
Transition	Move to the adjacent cell if it is inside the grid and not an obstacle
Step cost	1 per move
Goal test	Position = (4,4)
Heuristic	Manhattan distance $h(n) = \|r - 4\| + \|c - 4\|$, so $h(S) = 8$

Constraint analysis. Obstacles remove cells from the state space (6 of 25 cells, leaving 19 reachable cells). Four-direction movement with unit cost means that the minimum number of moves equals the optimal cost, and Manhattan distance never overestimates it (admissible and consistent).

Note on the algorithm. Plain BFS uses a FIFO queue and ignores heuristics. A priority queue ordered by $f(n) = g(n) + h(n)$ is the **A*** variant of the same search. Both are shown below so that the question's requirements (BFS, priority queue and Manhattan distance) are fully covered.

2.2 Part A – BFS Node Expansion (FIFO Queue)

Visited cells are marked when they are generated. Depth is shown as d.

Step	Expand (d)	New nodes added	Queue after expansion
1	(0,0) d0	(1,0), (0,1)	(1,0), (0,1)
2	(1,0) d1	(2,0)	(0,1), (2,0)
3	(0,1) d1	(0,2)	(2,0), (0,2)
4	(2,0) d2	(3,0)	(0,2), (3,0)
5	(0,2) d2	(1,2)	(3,0), (1,2)
6	(3,0) d3	(3,1)	(1,2), (3,1)
7	(1,2) d3	(2,2)	(3,1), (2,2)
8	(3,1) d4	(4,1), (3,2)	(2,2), (4,1), (3,2)
9	(2,2) d4	(2,3)	(4,1), (3,2), (2,3)
10	(4,1) d5	(4,2)	(3,2), (2,3), (4,2)
11	(3,2) d5	none ((4,2) already queued)	(2,3), (4,2)
12	(2,3) d5	(2,4)	(4,2), (2,4)
13	(4,2) d6	(4,3)	(2,4), (4,3)
14	(2,4) d6	(1,4), (3,4)	(4,3), (1,4), (3,4)
15	(4,3) d7	**(4,4) = G**	(1,4), (3,4), (4,4)
16	(1,4) d7	(0,4)	(3,4), (4,4), (0,4)
17	(3,4) d7	none	(4,4), (0,4)
18	(4,4) d8	**Goal test passed, stop**	(0,4)

BFS result: 18 nodes expanded. BFS finds the shortest path:

(0,0) → (1,0) → (2,0) → (3,0) → (3,1) → (4,1) → (4,2) → (4,3) → (4,4), with cost **8**.

2.3 Part B – Priority Queue with Manhattan Distance (A*)

The priority queue is ordered by f = g + h. Ties are broken by smaller h, then by earlier insertion.

Step	Expand (g, h, f)	Children inserted (g, h, f)	Priority queue after step
1	(0,0) (0, 8, 8)	(1,0) (1, 7, 8), (0,1) (1, 7, 8)	(1,0):8, (0,1):8
2	(1,0) (1, 7, 8)	(2,0) (2, 6, 8)	(2,0):8, (0,1):8
3	(2,0) (2, 6, 8)	(3,0) (3, 5, 8)	(3,0):8, (0,1):8
4	(3,0) (3, 5, 8)	(3,1) (4, 4, 8)	(3,1):8, (0,1):8
5	(3,1) (4, 4, 8)	(4,1) (5, 3, 8), (3,2) (5, 3, 8)	(4,1):8, (3,2):8, (0,1):8
6	(4,1) (5, 3, 8)	(4,2) (6, 2, 8)	(4,2):8, (3,2):8, (0,1):8
7	(4,2) (6, 2, 8)	(4,3) (7, 1, 8); (3,2) skipped, already queued with a lower g	(4,3):8, (3,2):8, (0,1):8
8	(4,3) (7, 1, 8)	(4,4) (8, 0, 8)	(4,4):8, (3,2):8, (0,1):8
9	(4,4) (8, 0, 8)	**Goal reached**	–

A* result: only **9 nodes expanded** (compared with 18 for BFS), because every expanded node has f = 8, which equals the true optimal cost.

2.4 Optimal Path (Visualised)

	c0	**c1**	**c2**	**c3**	**c4**
r0	S	⬜	⬜	⬛	⬜
r1	🟩	⬛	⬜	⬛	⬜
r2	🟩	⬛	⬜	⬜	⬜
r3	🟩	🟩	⬜	⬛	⬜
r4	⬛	🟩	🟩	🟩	G

🟩 = cells on the optimal path.

2.5 Result and Comparison

Measure	BFS (FIFO)	A* (Priority Queue + Manhattan)

Measure	BFS (FIFO)	A* (Priority Queue + Manhattan)
Nodes expanded	18	9
Path found	S → (1,0) → (2,0) → (3,0) → (3,1) → (4,1) → (4,2) → (4,3) → G	Same path
Optimal cost	**8**	**8**
Optimal?	Yes (unit costs)	Yes (admissible heuristic)

Final answer: the optimal path length is **8 moves** and the **optimal cost is 8**. Another path of equal cost exists through the top and right side: (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), G.

2.6 Algorithms in Pseudocode

Breadth-First Search (FIFO queue)

```
function BFS(start, goal):
  frontier <- FIFO queue containing start
  visited  <- {start}
  parent   <- {}
  while frontier is not empty:
    node <- frontier.dequeue()
    if node == goal: return PATH(parent, node)
    for each action in [Up, Down, Left, Right]:
      next <- RESULT(node, action)
      if next is inside the grid and not an obstacle and next not in visited:
        visited.add(next); parent[next] <- node
        frontier.enqueue(next)
  return FAILURE
```

A* Search (priority queue ordered by f = g + h)

```
function A-STAR(start, goal):
  frontier <- priority queue ordered by f; insert start with f = h(start)
  g[start] <- 0
  while frontier is not empty:
    node <- frontier.pop_lowest_f()        // ties: smaller h, then older entry
    if node == goal: return PATH(parent, node)
    for each valid neighbour next of node:
      new_g <- g[node] + 1
```

```
        if next not in g or new_g < g[next]:
            g[next] <- new_g; parent[next] <- node
            frontier.insert(next, new_g + h(next))
    return FAILURE
```

2.7 Heuristic, Distance and f-Value Maps

These maps make the behaviour of both algorithms visible at a glance. Obstacles are shown as ■.

Manhattan heuristic h(n) = |r − 4| + |c − 4|

	c0	**c1**	**c2**	**c3**	**c4**
r0	8	7	6	■	4
r1	7	■	5	■	3
r2	6	■	4	3	2
r3	5	4	3	■	1
r4	■	3	2	1	0

True shortest distance from the start, g*(n), found by BFS

	c0	**c1**	**c2**	**c3**	**c4**
r0	0	1	2	■	8
r1	1	■	3	■	7
r2	2	■	4	5	6
r3	3	4	5	■	7
r4	■	5	6	7	8

Evaluation f(n) = g*(n) + h(n)

	c0	**c1**	**c2**	**c3**	**c4**
r0	8	8	8	■	12

	c0	c1	c2	c3	c4
r1	8	■	8	■	10
r2	8	■	8	8	8
r3	8	8	8	■	8
r4	■	8	8	8	8

Reading the maps. 17 cells have f = 8, which equals the optimal cost, so they all lie on some shortest path. The cells (0,4) with f = 12 and (1,4) with f = 10 lie off every shortest path, and A* never needs to expand them. Because many cells tie at f = 8, the tie-break rule matters: choosing the smaller h among equal f values lets A* drive straight to the goal and expand only 9 nodes, whereas a poor tie-break could expand up to 17.

2.8 Step-by-Step Cost Table for the Optimal Path

Step	Move	Cell	g (cost so far)	h (estimate to goal)	f = g + h
0	Start	(0,0)	0	8	8
1	Down	(1,0)	1	7	8
2	Down	(2,0)	2	6	8
3	Down	(3,0)	3	5	8
4	Right	(3,1)	4	4	8
5	Down	(4,1)	5	3	8
6	Right	(4,2)	6	2	8
7	Right	(4,3)	7	1	8
8	Right	(4,4) = G	8	0	8

The value of f stays at 8 along the whole path. This happens when the heuristic is exactly correct on the route taken, so A* never has to reconsider its choice.

Admissibility. A heuristic is admissible if h(n) never exceeds the true remaining cost h*(n). On an empty grid with four-direction moves of cost 1, Manhattan distance is exactly the true cost. Obstacles can only lengthen the real route, so $h(n) \leq h^*(n)$ always holds.

Consistency. A heuristic is consistent if $h(n) \leq c(n, n') + h(n')$ for every move. One move changes $|r - 4| + |c - 4|$ by exactly 1 and costs 1, so $h(n) \leq 1 + h(n')$. With a consistent heuristic, A* never has to re-open a closed node, and the first time it removes the goal from the queue the path is optimal.

2.9 Scenario Analysis: Changing the Obstacles

The constraint "cannot pass through obstacles" drives the result. The following scenarios show how the optimal cost reacts.

Scenario	Effect on routes	Optimal cost	Search result
Original grid	Left route and top–right route are both shortest	8	Found
Extra obstacle at (3,1)	The left route is blocked at (3,0). The top–right route (0,1), (0,2), (1,2), (2,2), (2,3), (2,4), (3,4), G is unaffected	8	Found
Extra obstacle at (4,3)	The bottom route is blocked. The top–right route reaches G through (3,4)	8	Found
Extra obstacles at (3,1) and (2,2)	Left route blocked at (3,0). Top route stops at (1,2) because (2,2) is blocked, and (0,3) is already blocked	No path	BFS expands the 7 reachable cells (0,0), (1,0), (0,1), (2,0), (0,2), (3,0), (1,2) and reports failure

The last row shows that BFS is **complete**: when no path exists it terminates after exhausting the reachable region instead of looping forever.

2.10 Complexity and Comparison with Other Algorithms

Algorithm	Nodes expanded here	Path cost	Optimal?	Remark
BFS	18	8	Yes	Explores every cell closer than the goal first, ignoring direction
A* with Manhattan heuristic	9	8	Yes	Only cells on a shortest path are expanded, because the heuristic is accurate
UCS (Dijkstra)	About 18	8	Yes	With unit costs it behaves like BFS
DFS	Depends on move order	Not guaranteed	No	May find a longer path first
Greedy best-first	Few	Not guaranteed	No	Uses only h(n), so it can be misled by obstacles

The grid has 25 cells. At most 3 new neighbours are generated per expansion (the fourth is the cell just left), so BFS needs O(b^d) = O(3^8) in the worst case. The real count is only 18 because the grid is small and obstacles prune many branches. Memory use for BFS is the size of the frontier plus the visited set, and at most 25 cells are ever stored.

Question 3 – Autonomous Rescue Robot (Online Search with Risk Costs)

3.0 Problem Summary

A rescue robot starts at S in a grid of rooms and must reach a survivor at G. Some cells are blocked, risky zones carry extra cost, sensing is limited to adjacent cells, and the map is learned during the mission.

(a) Analysis of Constraints and Their Effect on Decision-Making

Constraint	Effect on the robot's decision-making	Priority

Constraint	Effect on the robot's decision-making	Priority
Blocked cells (obstacles)	They remove actions from the action set. Planning must treat these cells as impassable, and a plan that assumed a free cell must be repaired when the cell turns out to be blocked.	High (hard constraint)
Limited sensing (adjacent cells only)	The environment is partially observable. The robot cannot plan on the full map, so it keeps a belief or map memory and plans on what it knows.	High
Online knowledge update	Plan and act are interleaved. After each move the robot adds new percepts to its map and re-plans when something changes.	High
Risky zones (+2 cost)	Entering a risky cell costs 1 + 2 = 3. The cheapest path is no longer always the shortest, so a cost-sensitive search is needed.	Medium (soft constraint)
Minimise total cost with safe navigation	The objective function is the sum of step costs. Safety is built into the cost model and is never traded away silently.	Medium (objective)
Moves: up, down, left, right	Branching factor is at most 4, and actions are reversible, so the robot can back out of a dead end.	Low (structure)

Prioritisation: (1) hard feasibility (obstacles and safety), (2) information limits (partial observability), (3) cost optimisation (risk and distance). Hard constraints are never violated, and soft constraints are minimised.

(b) Solution Approach: Online Uniform Cost Search with Re-planning

Since step costs differ (1 for normal cells, 3 for risky cells), plain BFS is not suitable. The approach is **Uniform Cost Search (UCS) run on the robot's known map**, repeated during an **online search** loop. An admissible heuristic such as Manhattan distance can be added to make it A*.

Cost model

Cell type	Cost to enter
Normal cell	1
Risky zone	1 + 2 = 3
Obstacle	Not allowed (∞)
Unknown cell	Assumed free with cost 1 (optimistic), corrected once sensed

Algorithm steps

1. **Initialise:** the robot starts at S with a map memory containing only S.
2. **Sense:** observe the four adjacent cells and record them as free, blocked or risky.
3. **Update knowledge:** add the new percepts to the known map.
4. **Plan:** run UCS (priority queue ordered by path cost g) over the known map, treating unknown cells as free, to find the lowest-cost path to G.
5. **Act:** execute **only the first move** of the plan, then move one cell.
6. **Check:** if G is reached, stop. Otherwise go back to step 2.
7. **Repair:** if a planned cell turns out to be blocked or risky, update the map and re-plan from the current cell. The robot can backtrack, since moves are reversible.

```
function RESCUE-ROBOT(start, goal):
  known_map <- {start}
  position  <- start
  while position != goal:
    percepts   <- SENSE(adjacent cells of position)
    known_map  <- UPDATE(known_map, percepts)
    plan       <- UNIFORM-COST-SEARCH(known_map, position, goal)
    if plan is empty: return FAILURE   // no known safe route
    position   <- MOVE(first action of plan)
  return SUCCESS
```

Illustration of the cost trade-off. Using the same 5×5 layout as Question 2 with two assumed risky cells, 🟧 at (2,2) and (2,3):

	c0	**c1**	**c2**	**c3**	**c4**
r0	🤖 S	⬜	⬜	⬛	⬜

	c0	c1	c2	c3	c4
r1	⬜	⬛	⬜	⬛	⬜
r2	⬜	⬛	🟧	🟧	⬜
r3	⬜	⬜	⬜	⬛	⬜
r4	⬛	⬜	⬜	⬜	🏁 G

Candidate route	Cost calculation	Total cost
Safe route: (1,0), (2,0), (3,0), (3,1), (4,1), (4,2), (4,3), G	8 moves × 1	**8**
Risky route: (0,1), (0,2), (1,2), (2,2) 🟧, (2,3) 🟧, (2,4), (3,4), G	1 + 1 + 1 + 3 + 3 + 1 + 1 + 1	**12**

Both routes have 8 moves, but the risky route costs 12. UCS selects the safe route (cost 8), which is how the cost function protects the robot. If the robot first explores the top route and senses the risky cells, the updated map raises that route's cost and the re-planning step switches it to the safe route.

(c) Application of AI Concepts

1. Intelligent agent (PEAS description)

PEAS	Description
Performance measure	Reach the survivor G, minimise total cost (steps + risk penalties), avoid hazards
Environment	Disaster-affected building modelled as a grid of rooms with obstacles and risky zones
Actuators	Movement: Up, Down, Left, Right
Sensors	Proximity sensors that detect adjacent cells, plus position tracking

The robot is a **goal-based, utility-oriented agent with a learning (map-building) component**. It keeps an internal model (known map), has a goal (reach G), and chooses actions by cost.

2. Rationality

A rational agent selects the action expected to maximise its performance measure, given its percept history and built-in knowledge. In this case:

- It reaches the goal with the **minimum total cost** on its known map (UCS optimality).
- It **avoids unnecessary risky cells** unless the detour costs more than the penalty.
- It **uses its percept history** (the map memory), so it does not repeat mistakes.
- Rationality is **bounded by information**. The robot is rational with respect to what it can sense, not omniscient.

3. Environment type

Property	Classification	Reason
Observability	**Partially observable**	Only adjacent cells are sensed
Agents	**Single agent**	Only the robot acts (survivors are static)
Determinism	**Deterministic** (assumed)	A move leads to a predictable next cell
Episodic / Sequential	**Sequential**	Current moves affect future cost and options
Static / Dynamic	**Static** (assumed)	Building layout does not change during planning
Discrete / Continuous	**Discrete**	Grid cells, finite actions
Known / Unknown	**Unknown (initially)**	The map is learned during the mission

(d) Justification of the Chosen Approach

Alternative	Why it is less suitable
BFS	Treats all steps as equal, so it can return a path through risky cells that is not cost-optimal.
DFS	Not optimal and may take very long or unsafe routes.
Offline UCS / A* only	Needs the full map in advance, which the robot does not have.

Alternative	Why it is less suitable
Greedy search	Ignores accumulated risk cost and can be misled into hazardous zones.
Online UCS with re-planning (chosen)	Handles variable costs, partial observability and dynamic map updates together.

Reasons for choosing it

1. **Variable step costs:** UCS is optimal when costs differ (1 vs 3), unlike BFS.
2. **Partial observability:** the interleaved sense, plan and act loop works without a full map.
3. **Safety:** risky cells are priced into the path cost, and obstacles are never entered.
4. **Completeness:** UCS is complete (positive step costs) and optimal on the known map. Reversible moves allow recovery from dead ends.
5. **Extensibility:** adding Manhattan distance turns it into A*, which reduces node expansion without losing optimality, as shown in Question 2 (9 vs 18 expansions).

3.1 Formal Problem Formulation (Online Search)

In **offline search** the agent computes a complete plan before it moves. In **online search** it interleaves planning and acting, because it can only learn about the world by moving through it. The rescue problem is an online search problem.

Element	Definition for the rescue robot
State	The robot's cell plus its knowledge map (status of every cell seen so far)
Initial state	Robot at S, map containing only what the sensors show from S
Actions	Up, Down, Left, Right, available only where the neighbouring cell is not known to be blocked
Transition	Move to the neighbouring cell; sense the new neighbours
Step cost	1 for a normal cell, 1 + 2 = 3 for a risky cell
Goal test	Robot cell = G (survivor reached)
Performance measure	Total path cost actually travelled, compared with the best offline

Element	Definition for the rescue robot
	cost

An online algorithm is judged by its **competitive ratio**, which is the cost it actually pays divided by the cost of the optimal path it would have chosen with full knowledge of the map.

$$\text{competitive ratio} = \frac{\text{cost paid by the online robot}}{\text{optimal cost with full map knowledge}}$$

3.2 Knowledge Representation

The robot stores one label per cell. Labels only change in one direction, from Unknown to a sensed value.

Label	Meaning	Cost used by the planner
Unknown	Never sensed	1 (optimistic assumption, called the free-space assumption)
Free	Sensed, no hazard	1
Risky	Sensed, hazard present	3
Blocked	Sensed, impassable	Not allowed
Visited	The robot has stood there	Its own cost, kept for loop checks

Assuming that unknown cells are free is **optimistic**, so the planned cost never overestimates the true cost. This keeps the planner guided toward the goal, and a wrong assumption is corrected by re-planning as soon as the sensors reveal the truth.

3.3 Worked Online Simulation

Both simulations use the 5×5 layout from section (b), where 🟧 marks the risky cells (2,2) and (2,3). The robot knows the grid size and the goal cell, senses only its four neighbours and plans with UCS plus the Manhattan heuristic. The two runs differ only in how a tie between equal-cost plans is broken.

	c0	**c1**	**c2**	**c3**	**c4**
r0	🤖 S	⬜	⬜	⬛	⬜
r1	⬜	⬛	⬜	⬛	⬜
r2	⬜	⬛	🟧	🟧	⬜

	c0	c1	c2	c3	c4
r3	☐	☐	☐	■	☐
r4	■	☐	☐	☐	▩ G

Run A: ties broken in favour of moving Down first

Step	Robot at	New percepts	Re-plan?	Planned remaining route (estimate)	Action	Cost so far
0	(0,0)	(1,0) free, (0,1) free	Initial plan	(1,0), (2,0), (3,0), (4,0), (4,1), (4,2), (4,3), G (8)	Down	1
1	(1,0)	(2,0) free, (1,1) blocked	No	(2,0), (3,0), (4,0), … (7)	Down	2
2	(2,0)	(3,0) free, (2,1) blocked	No	(3,0), (4,0), … (6)	Down	3
3	(3,0)	**(4,0) blocked**, (3,1) free	**Yes**	(3,1), (4,1), (4,2), (4,3), G (5)	Right	4
4	(3,1)	(4,1) free, (3,2) free	No	(4,1), (4,2), (4,3), G (4)	Down	5
5	(4,1)	(4,2) free	No	(4,2), (4,3), G (3)	Right	6
6	(4,2)	(4,3) free	No	(4,3), G (2)	Right	7
7	(4,3)	(3,3) blocked, G visible	No	G (1)	Right	**8**

Run A reaches the survivor with **cost 8** after one re-plan, at step 3 where the blocked cell (4,0) is discovered. No risky cell is entered, and the competitive ratio is 8 / 8 = **1.0**.

Run B: ties broken in favour of moving Right first

Step	Robot at	New percepts	Re-plan?	Planned remaining route (estimate)	Action	Cost so far
0	(0,0)	(1,0) free, (0,1) free	Initial plan	(0,1), (0,2), (0,3), (0,4), (1,4), (2,4), (3,4), G (8)	Right	1

Step	Robot at	New percepts	Re-plan?	Planned remaining route (estimate)	Action	Cost so far
1	(0,1)	(1,1) blocked, (0,2) free	No	(0,2), (0,3), … (7)	Right	2
2	(0,2)	**(0,3) blocked**, (1,2) free	**Yes**	(1,2), (2,2), (3,2), (4,2), (4,3), G (6)	Down	3
3	(1,2)	(1,1) blocked, (1,3) blocked, **(2,2) risky**	**Yes**	Through (2,2): 3 + 4 = 7. Retreat to S and use the left route: 3 + 8 = 11. Chosen: through (2,2)	Down (enter risky cell)	6
4	(2,2)	(2,1) blocked, (2,3) risky, (3,2) free	No	(3,2), (4,2), (4,3), G (4)	Down	7
5	(3,2)	(3,1) free, (3,3) blocked, (4,2) free	No	(4,2), (4,3), G (3)	Down	8
6	(4,2)	(4,1) free, (4,3) free	No	(4,3), G (2)	Right	9
7	(4,3)	(3,3) blocked, G visible	No	G (1)	Right	**10**

Run B reaches the survivor with **cost 10**, because the robot walked into a dead-end corridor and had to pay the risky-cell penalty once. At step 3 the planner compared the two options with its updated map and chose the cheaper one, which is rational behaviour. The competitive ratio is 10 / 8 = **1.25**.

Comparison of the outcomes

Outcome	Total cost	Risky cells entered	Re-plans	Competitive ratio
Offline optimum (full map known)	8	0	–	1.00
Online Run A (Down first)	8	0	1	1.00
Online Run B (Right first)	10	1	2	1.25

Outcome	Total cost	Risky cells entered	Re-plans	Competitive ratio
Run B with risky cells forbidden (hard safety rule)	14	0	2	1.75

The last row works out as follows: the robot pays 3 to reach (1,2), pays 3 to walk back to S, then pays 8 for the safe left route, giving 3 + 3 + 8 = 14. Forbidding risky cells entirely is therefore more costly than pricing them.

3.4 Properties of the Online Strategy

Property	Result	Reason
Completeness	Yes, if the space is finite and safely explorable	Moves are reversible, so the robot can always retreat from a dead end
Optimality	Optimal on the known map at each step, but not guaranteed optimal overall	Future cells are unknown, so early choices can be wrong (Run B)
Planning cost per step	O(N log N) for N known cells	One UCS run with a priority queue
Total planning cost	$O(N^2 \log N)$ in the worst case	At most one re-plan per move, and the grid has N = 25 cells
Memory	O(N)	One label per cell
Sensor use	Four neighbours per step	Matches the limited-sensing constraint

3.5 Risk Weight and Safety Variants

The penalty for a risky cell is a design parameter. Let w be the extra cost of entering a risky cell, so that its total cost is 1 + w. The question sets w = 2.

At step 3 of Run B the robot compares entering the risky cell with retreating:

$$\underbrace{(1 + w) + 4}_{\text{enter (2,2) and continue}} \;\le\; \underbrace{3 + 8}_{\text{retreat and take left route}} \quad\Longrightarrow\quad w \le 6$$

Risk penalty w	Cost of entering (2,2)	Cost of retreating	Rational choice

Risk penalty w	Cost of entering (2,2)	Cost of retreating	Rational choice
0	5	11	Enter
2 (given)	7	11	Enter
6	11	11	Indifferent
10	15	11	Retreat

When the penalty is small the robot accepts a short risky stretch, and when the penalty grows beyond 6 it prefers a longer safe route. This shows how the cost function turns the "safe navigation" requirement into a measurable trade-off.

Safety variants for a real rescue deployment

Variant	Change to the model	Effect
Hard safety	Treat risky cells as blocked ($w = \infty$)	Never enters danger, but may fail if the survivor is only reachable through a risky zone
Soft safety (this report)	Risky cells cost $1 + w$	Balances speed and safety, and always finds a route if one exists
Time-critical rescue	Add a penalty per step so that shorter routes matter more	Favours speed when the survivor's condition is urgent
Dynamic hazards	Re-sense cells already visited and update costs	Handles collapses and spreading fire, at the price of more sensing
Noisy sensors	Keep a probability for each cell instead of a fixed label	Needs a belief-state agent and expected-cost planning

Other online algorithms. Learning Real-Time A* (LRTA*) updates a heuristic value for each visited cell and is suited to unknown maps with no stored plan, but it can revisit cells many times. Online DFS with backtracking explores every cell systematically but ignores cost. Uniform cost search with re-planning, as used here, gives the lowest travelled cost on average for this problem, because it responds directly to both obstacles and risk penalties.

Overall Conclusion

Question	Technique	Result
Q1	Backtracking search (CSP)	D1 – Morning, D2 – Afternoon, D3 – Night
Q2	BFS and A* (Manhattan heuristic)	Optimal path cost = 8 (BFS expands 18 nodes, A* expands 9)
Q3	Online Uniform Cost Search with re-planning	Safe, cost-minimising navigation under partial observability

Prepared by D. Jeshurun Silas (Reg. No. 192525324) for Assessment Tool 2, Artificial Intelligence (CSA17).

Comparative Summary of Techniques

Aspect	Q1: Backtracking (CSP)	Q2: BFS and A*	Q3: Online UCS with re-planning
Problem class	Constraint satisfaction	Shortest path, full knowledge	Shortest safe path, partial knowledge
Environment	Fully observable, static	Fully observable, static, deterministic	Partially observable, sequential, initially unknown
Main data structure	Recursion stack and domains	FIFO queue (BFS), priority queue (A*)	Priority queue plus knowledge map
Optimality	Any valid schedule (2 exist)	Optimal path cost 8	Optimal on the known map, ratio 1.0 to 1.25 in the runs
Key strength	Prunes invalid branches early	Heuristic cuts expansions from 18 to 9	Adapts to new information
Key limitation	Exponential in the worst case	Needs the whole map in advance	Early choices may be wrong

Reflection and Learning Outcomes

What was learned

- A problem becomes solvable once its states, actions, constraints and costs are written down precisely.
- Constraint order matters: cheap unary checks first, then binary checks, saves effort.
- Propagation (forward checking and arc consistency) can remove backtracking altogether on small problems.
- A good heuristic changes the work done but not the answer: A* and BFS both give cost 8, with 9 versus 18 expansions.
- Online search accepts that the plan may change. The measure of quality is the competitive ratio, not only the final path.

Course outcome CO1 mapping (BL3: Apply)

CO1 element	Evidence in this report
Apply intelligent agent concepts	PEAS description, agent types and rationality analysis in Q3(c) and the Introduction
Apply uninformed search methods	BFS and UCS traces in Q2 and Q3, with the comparison table in the Introduction
Solve problems effectively	Verified results: schedule D1 M / D2 A / D3 N, path cost 8, rescue cost 8 to 10

Self-check against the rubric

Criterion	Evidence	Self-rating
Problem understanding (25%)	Full formulations, assumptions and constraint tables for all questions	Level 4
Constraint analysis (25%)	Priorities, redundancy of C4, obstacle and risk scenarios	Level 4
Solution development (25%)	Complete traces with verified, optimal results	Level 4
Application of concepts (15%)	CSP propagation, admissibility proof, online search	Level 4

Criterion	Evidence	Self-rating
	properties	
Justification (10%)	Comparisons with alternative methods and threshold analysis	Level 4

Glossary

Term	Meaning
Admissible heuristic	An estimate that never exceeds the true remaining cost
Arc consistency	Every value of a variable has a compatible value in each related variable
Backtracking	Depth-first assignment that undoes a choice when a constraint fails
Branching factor (b)	Number of successors generated from a node
Competitive ratio	Cost paid by an online algorithm divided by the optimal offline cost
Consistent heuristic	Estimate that obeys $h(n) \leq c(n, n') + h(n')$ for every move
Constraint satisfaction problem	Variables, domains and constraints with the goal of a complete valid assignment
Forward checking	Removing values from unassigned variables that conflict with a new assignment
Frontier	The set of generated nodes waiting to be expanded
Heuristic	A function that estimates the remaining cost to the goal
Manhattan distance	Sum of horizontal and vertical distances between two cells
MRV	Minimum remaining values: pick the variable with the smallest domain
Online search	Planning and acting are interleaved because the map is

Term	Meaning
	learned while moving
PEAS	Performance measure, Environment, Actuators, Sensors
Rational agent	An agent that acts to maximise its expected performance measure

References

1. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th edition. Pearson, 2020. Chapters on intelligent agents, solving problems by searching, and constraint satisfaction problems.
2. Hart, P. E., Nilsson, N. J. and Raphael, B. "A Formal Basis for the Heuristic Determination of Minimum Cost Paths." *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 1968, pp. 100–107.
3. Korf, R. E. "Real-Time Heuristic Search." *Artificial Intelligence*, 42(2–3), 1990, pp. 189–211.
4. Mackworth, A. K. "Consistency in Networks of Relations." *Artificial Intelligence*, 8(1), 1977, pp. 99–118.